

Trust, but Replay: Auditing Published Mathematical Claims

אם ירצה השם

Daniyel Yaacov Bilar, Chokmah LLC

[ORCID 0000-0002-9040-6914](https://orcid.org/0000-0002-9040-6914), chokmah-dyb@pm.me DOI: [10.5281/zenodo.22926996](https://doi.org/10.5281/zenodo.22926996)

Companion methods paper to the fragile-proof-audit campaign. Exported 2026-09-23.

י"ב תשרי תשפ"ז

Abstract

Published mathematics is trusted far more than it is independently replayed. Peer review checks reasoning, not computation - and increasingly, part of the proof *is* a computation. This paper describes a short, intensive audit campaign (six calendar days, 2026-09-18 through 2026-09-23) that replays published mathematical claims from scratch under a hostile prior: every locked gate is built to produce the opposite verdict where the opposite is correct (the four exact-equality gates are exempt - Sec. 4.2), and a CI-enforced verdict lock fails loudly on drift in either direction. The contribution is the playbook, not the verdicts: a five-disposition taxonomy (BREAK, GAP, PASS, SKIP, UNKNOWN) with a polarity rule separating gate verdicts from claim-level dispositions; seven attack types classified by mechanism, each with a worked example; and the evidentiary disciplines - paper-first gating, discrimination controls, gate-before-prove, fail-closed replay, independent anchors - together with the record of where those disciplines were violated and what caught the violations. The 25 locked gates (17 BREAK / 8 PASS) fall into three strata with very different evidentiary weight: four historical calibrations, twelve live-literature targets, and eight low-stakes preprints, plus one infrastructure oracle. Case studies include a refutation replay, an execution-verified confirmation, a canonization, and an unfinished formalization line; the limitations section states what no discipline closes. The campaign repository is private at the time of writing; at submission it will be mirrored publicly, with the submitted version citing its exact commit SHA and archival [DOI](#). Gates refute routes, not theorems.

1. Introduction: the verification gap

Peer review checks reasoning, not computation. A referee reads the argument, follows the lemmas, and judges whether the inferences hold. What the referee almost never does is replay the computation: the thousand-line script, the supplementary data file, the certificate the theorem's truth depends on. Those artifacts are trusted on the strength of the prose around them - and prose is not a checksum.

The rot is ordinary, not scandalous. Authors move institutions and their pages die; file formats age out; a dataset lives on a personal site with no mirror. In the q-TSPP line, the authors' 293 MB certificate archive survived only on the Wayback Machine; the live site was gone. It had to be re-read, re-hashed, and replayed before a single lemma could be formalized. Nothing about that was adversarial. It was still most of the work.

The campaign described in this paper takes the next step: it treats published computational claims under a *hostile prior*. Not because authors are dishonest - the overwhelming majority of the defects found here are mistakes, not misconduct - but because trust is not a verification method. Every locked gate was replayed from pinned inputs by a script built to produce the opposite verdict where the opposite was correct; GAP, SKIP, and UNKNOWN dispositions have no executable gate by definition (section 2). Where the claim survived, that is recorded as a PASS with the same weight as a BREAK: the method does not grade on a curve.

Two scoping admissions, then a word on the labor. First, the timeline is short - the campaign's first commit is 2026-09-18. The claim of this paper is the reproducibility of the *method*, not the longevity of the results: every gate, control, and certificate in the repository can be replayed by a reader, and the verdict lock (Sec. 2) fails loudly if any of them drifts. Second, the targets were selected for fragility by harvest dossiers, not sampled at random. This paper makes no claim about the base rate of defective proofs in the literature. It claims only that *these* routes were tested, *this* is how, and *these* are the dispositions - with the failures of the method itself recorded alongside (Sec. 4.7).

Third, the labor. The throughput above - 25 gates, four replay lanes, a 91-chunk canonization, a two-milestone Lean formalization, and eight prose audits in six calendar days (2026-09-18 to 2026-09-23) - was produced by AI agents working under human direction: they wrote the gates, ran the computations, drafted the audit notes, and assembled this paper; the human set the targets, reviewed the verdicts, and owns every disposition. Target curation used the same division of labor: the human wrote deep-research prompts and ran them on the Kimi 3 and Grok 3.1 deep-research tools to assemble the candidate harvest list, then shortlisted it by hand. Nothing else predates the repository's 2026-09-18 root commit. This is stated plainly because it is load-bearing, not confessional. An agent pipeline fails in characteristic ways (Sec. 4.6--Sec. 4.7), and Sec. 4's disciplines are in large part the scar tissue from those failures. The verification-of-verification question, *who audited the auditors*, is answered the same way this paper answers everything else: by the artifact. Every gate is a script a reader can run; every verdict is pinned in a lock that fails loudly on drift; every failure of the method is published beside its successes. Trust the replay, not the resume - including ours.

That review was a full replay, not a skim. The author directed a clean replay of the twenty-five-gate verdict lock (executed by an AI agent at the author's direction) - 25/25 [ok], zero drift - then re-read every audit note and every prose disposition, checking each verdict against the underlying evidence rather than the recorded pin, and spot-checked gate sources against their meta receipts. The replay caught two missing Python dependencies in the review environment (`pypdf` , `networkx`); three gates aborted fail-closed on their PDF-identity controls until the dependencies were installed, then went green. Approximately two hours of review found no verdict discrepancies. The replay log is pinned in the repository.

The rest of the paper is the playbook. Sec. 2 defines the disposition taxonomy, Sec. 3 the attack types by mechanism with one worked example each, Sec. 4 the evidentiary disciplines and the record of their violations, Sec. 5 the case studies, Sec. 6 the limitations.

2. The disposition taxonomy

Every audit in the campaign ends in exactly one disposition. The taxonomy exists for one reason: verdicts drift. Without fixed categories, a "the proof has a gap" quietly becomes "the theorem is false" in retelling, a "we couldn't check this" becomes "it passed," and a refuted lemma becomes a refuted mathematician. Each disposition below is defined by what was *done*, not by how bad the news is, and each carries a scope rule about what it does and does not say.

The five dispositions: **BREAK, GAP, PASS, SKIP, UNKNOWN**.

BREAK - the route is refuted by a gated, controlled witness

A BREAK means: a specific lemma, as the paper states it, is false - exhibited by a gate with a passing discrimination control (Sec. 4.2). The record is always lemma, instance, false instance. "Gates refute routes, not theorems" (Sec. 3): $\zeta(5)$ is probably irrational, and Goldbach's conjecture is untouched by the semi-continuous (S.C.E.) audit - what died in each case is a specific chain of lemmas, and the audit note names them.

A BREAK is on the verdict lock (see below): the gate that produced it runs in CI (continuous integration), and any drift in its verdict fails the build.

GAP - the inference is broken, but the statement is not refuted

A GAP means: the paper's *argument* does not establish its conclusion, but no lemma statement was falsified. The defect is found by a prose audit, not by computation - typically a quantifier error, an undischarged hypothesis, or an invalid analytic inference - and the residue is not finitely gateable, so there is no executable gate to lock. The route as written is refuted; the theorem is untouched; and a GAP is explicitly *not* a BREAK.

Three instances from the campaign: FRK-UC (a quantifier error plus a definition applied outside its domain), LEG-NS (a contour integral claimed analytic on a neighborhood it cannot reach), and the Erdos--Straus Theorem-10 interval (the exhibited parameters fail, but witnesses exist in the same construction family) - a GAP can be *repairable*, and the disposition records that, because "the route as written fails" and "the route cannot be repaired" are different claims.

GAPs are not on the verdict lock: there is no gate to pin, only the audit note and its verbatim quotations.

The GAP evidentiary standard. A GAP is a prose verdict with no executable gate and no kernel, so it carries a fixed standard, adopted 2026-09-23: (1) the exact failing sentence, equation, or inference quoted verbatim with its location; (2) the gap re-derived independently of the audit note; (3) repairability judged explicitly, with the repair exhibited or the obstruction named; (4) the evidence pinned (paper version, script paths, replay exit codes). Independence in (2) is operational, not honorific: the re-deriver - a different model or a human - receives the paper and the quoted sentence but not the audit note's diagnosis. A fresh instance of the same pipeline re-deriving the same error is correlation, not confirmation. FRK-UC and LEG-NS were re-examined under this standard on 2026-09-23 and both survived; the Erdos--Straus GAP already met it - and the standard caught a real error in its own note, the false "every other even k fails" claim, narrowed to the checked cases. A standard that catches errors in our own notes is doing its job.

PASS - the gate's claim is independently confirmed

A PASS means the gate confirmed what it was built to test, within the scope its audit note records. PASS escalates (a confirmed lemma can be built on); it is never forced (a gate that cannot fail is not run, Sec. 4.2). The Jacobian-2D campaign is the example: the coefficient identities passed their computational gates *and* the proof text passed its prose audit, so the target is either correct or fails outside our gates' reach. A PASS is a statement about the audit's reach, not a certificate of truth.

Gate-level PASS verdicts are on the verdict lock exactly like BREAKs (see below). Off-lock prose PASSes - a route judged sound without a runnable gate, as in Appendix C's JAC-2D type-G and Jana--Karmakar type-C entries - are recorded separately and carry no lock protection.

SKIP - not auditable as stated

A SKIP means the target, as stated, admits no gate: nonconstructive arguments, claims with no witness to exhibit, routes whose decisive step cannot be instantiated at any parameters. SKIP is a verdict about the audit, not the paper - "we cannot test this," not "this is wrong." The NCI (Non-Cancelling Intersections; Wilhelm, arXiv:2608.27416v2) target is the example: skipped because the alleged finite gate was absent from the paper as written.

UNKNOWN - artifacts verified, kernel not re-checkable

An UNKNOWN means the checkable parts check out but the decisive verification step cannot be replayed on our toolchain. The quantum Hedetniemi audit verified the Python certificates and the source catalog (PASS on those lanes) but the kernel step requires the authors' Lean 4.19.0 environment, which we do not run. UNKNOWN is the honest alternative to inflating a partial check into a confirmation - or discarding it.

The polarity rule

A verdict attaches to the *gate's claim*, not to the paper's conclusion. The Lame 1847 audit is the canonical case: the locked gates are **PASS** - the class number $h(\mathbb{Q}(\zeta_{23})) = 3$ is confirmed - and it is precisely that confirmed fact that kills Lame's route, which needed the class number to be 1. The route verdict is BREAK; the gate verdicts are PASS; confusing the two would invert the meaning of the evidence. Every audit note states both polarities separately.

The verdict lock

`scripts/gates/check.py` pins every gate's expected verdict in `EXPECTED_VERDICT` - currently 25 gates (17 BREAK / 8 PASS). The lock runs in CI; drift in either direction fails the build. It is not an all-BREAK lock, and it was a mislabel to ever call it one: a PASS that stops passing is as much a drift as a BREAK that stops breaking. GAPS, SKIPS, and UNKNOWNs are not on the lock - they are prose dispositions recorded in `docs/audits/`, not executable gates - and neither are the Gomila confirmation or the Polya canonization, which are not verdicts about routes at all.

Append-only history

The campaign's history is append-only: git plus dated WORKPLAN checkpoints. When a verdict is corrected, the prior verdict is marked superseded, not deleted. The odd-zeta audit's early " Λ_m unevaluable" verdict was superseded by the Lemma 5.1 BREAK on 2026-09-20; both remain visible, with the correction dated. Summary tables show the current verdict with a "superseded" note: a taxonomy that rewrites its past cannot be trusted about its present.

What the catalog represents: stratification by community standing

The lock's 25 gates are not 25 draws from "published mathematics": selected for fragility (Sec. 1), they fall into three strata with very different evidentiary weight. Pooled, they invite the base-rate misreading this paper disclaims; stratified, each stratum carries only its own evidentiary weight:

- **Historical calibration (4 gates: 2 BREAK / 2 PASS).**
Kempe--Fritsch, Tait--Tutte, and Lame's 1847 cyclotomic route (two gates). Famous, long-settled failures replayed to validate the gates. A BREAK here confirms the gate can hit a known target; it says nothing about the literature.
- **Live literature (12 gates: 8 BREAK / 4 PASS).** Claims from the active research literature, thirteen papers across twelve gates (`rr_qexpand` covers both Lau--Ono and

Huang--Lau--Ono--Paule). Of the eight BREAKs - one refutes a claimed proof with no prior published refutation, Sun's Catalan route (arXiv:2609.04176, `cat_g`) - and seven are counterexamples to stated conjectures, each already refuted in print: Baste et al.'s domination conjecture (`baste_domination`), Tang--Zhang, Chung--Graham--Spiro (2020), Sarkozy's Conjecture 65, Thakur (2015), Salez--Youssef's Conjecture 1, and Cohen's Conjecture 66. Where a prior published refutation exists it is cited in the audit note (Zeng--Liu--Ratnavelu for Tang--Zhang, Aliabadi for Chung--Graham--Spiro, Tang for Sarkozy, Giraudin for Thakur, Munch for Salez--Youssef, Ibarra for Cohen, Afrasyab for Baste); the gates compute their witnesses from the pinned claim artifact rather than replaying the refutation's argument. The four PASS gates are `mah_3` (claim-level SKIP - Sec. 6.2), `es_cover` (Lopez), `rr_qexpand`, and `pdn1` (Du--Yao). This stratum is where a BREAK is evidence about published mathematics - and where the PASSes show the gates confirm as well as refute.

- **Low-stakes preprints (8 gates: 7 BREAK / 1 PASS).** Withdrawn, crank-adjacent, or never-trusted claims: Suman's $\zeta(5)$ (withdrawn), the Preprints.org odd-zeta manuscript, the Goldbach S.C.E. model, Agama's twin-prime area method, two Gnanang preprints, Ghermoul's Erdos--Straus equation, and Su's 43-version 2D-Jacobian claim (PASS - the computational gate passed, and a separate off-lock type-G prose audit passed too). BREAKs here are training kills: they exercise the machinery on targets nobody relied on.

(`giuga_oracle`, a standing PASS, is infrastructure - the known-Giuga-number oracle other gates consult - not a claim audit.)

The honest reading: the campaign's weight as evidence about the literature rests on the middle stratum - eight BREAKs against live claims (one against a claimed proof, seven against stated conjectures), three live claims fully surviving their gates. The low-stakes stratum proves the machinery runs; the historical stratum proves it aims true. Neither is presented as more than that.

3. The attack types (A--G)

The campaign's gates are classified by *mechanism*, not by subject area. Each type below names a route a proof can take, the gate built to test it, and one worked example from the audit catalog (`docs/audits/`, `docs/blueprint/`). The standing doctrine throughout:

Gates refute routes, not theorems. $\zeta(5)$ is probably irrational; FLT is true; the four-color theorem is true. What dies is a specific lemma chain, recorded as lemma, instance, false instance.

The taxonomy is append-only: a new mechanism gets a new letter.

A - Scalar gate

Route attacked. Claims whose decisive content is a numerical statement about a specific constant - typically "best constant" conjectures, where the entire proof stands or falls on one inequality.

Gate construction. Compute the constant, or an explicit witness violating it, by two independent arithmetics (exact rational algebra and high-precision floating point), with the violation margin far above any arithmetic noise floor. The discrimination question here is not "does it fire on everything" but "is the witness typical or extremal" - answered by sampling the neighborhood.

Worked example. Tang--Zhang Schatten-norm constant (`docs/audits/tang-zhang-schatten.md`; refutation artifact Zeng--Liu--Ratnavelu, arXiv:2608.15558, Theorem 1.1). This BREAK independently confirms Zeng--Liu--Ratnavelu's refutation: the gate re-derives the violation from scratch along two independent computation paths rather than replaying their argument. The conjectured best constant $C_{p,m}^{TZ} = \sqrt{x(x+m-1)}/(x^p+m-1)^{1/p}$ - with x the unique $x > 1$ solving $x^p - 2x - (m-1) = 0$ - is exceeded by an explicit rank-one pair at $p = 3/2$, $m = 2$: $R = 1.03641365870489 \dots > 207/200 > C_{3/2,2}^{TZ} = 1.03465395185143 \dots$. Two paths - exact rational

Gram-matrix algebra and 60-digit mpmath - agree to $1e-40$; margins $\sim 1.4e-3$ above and $\sim 3.5e-4$ below against a $1e-20$ safety floor. Control: 5,000 random rank-one pairs at the same (p, m) - only $\sim 1\%$ exceed the conjectured constant and the witness sits near the true extremum, so the gate is discriminating, not trigger-happy. **Verdict:** **BREAK.**

B - Base-case kill

Route attacked. Induction or irrationality arguments whose load-bearing step is a finiteness / non-existence claim at the base - the step the rest of the proof assumes without exhibiting.

Gate construction. Instantiate the base case exactly, with all constraints, and solve it. If solutions exist, the induction never starts and nothing downstream matters.

Worked example. Suman $\zeta(5)$ (`docs/audits/suman-zeta5.md`). Eq. (48) is claimed to have no integer solutions; it is the induction base for Theorem 1. At $n = 1$, $d_1 = \text{lcm}(1) = 1$, constraints $0 \leq k \leq d_1$ and $d_1 \mid kb$: solutions $(a, b, k) = (2, 1, 0)$ ($a = 2b$) and $(1, 1, 1)$ ($a = b$) exist. Suman dismisses these because they "would force $\zeta(5) \in \{1, 2\}$ " - but solvability of Eq. (48) is independent of whether $\zeta(5)$ is an integer. **Verdict:** **BREAK.**

C - WZ-certificate audit (WZ = Wilf--Zeilberger)

Route attacked. Claims resting on a Wilf--Zeilberger certificate pair - the "a rational function proves the identity" genre, where the decisive object is a machine-produced certificate the reader is expected to trust.

Gate construction. Independently re-verify the certificate pair by exact evaluation on a dense grid. The known trap is Pochhammer conventions at negative indices: $(a)_{-n} = (-1)^n / (1-a)_n$ must be enforced, not assumed.

Worked example. Jana--Karmakar (arXiv:2501.10109). The claimed WZ pair survived 630 + 630 exact telescoping checks (Lemmas 2.1 and 3.1) and 96 checks of the summed theorems - the audit produced a PASS. The audit still proved its worth: the first harness produced 66 false mismatches by omitting the $(a)_{-n}$ convention above; that convention is now enforced in `scripts/harness/pochhammer.py`. A type-C gate that declines to fire is the discipline working as designed.

D - Finite q-expansion

Route attacked. Partition congruences and modular-equation claims whose decisive content is exact identities between q-series - checkable term by term, but only to the depth actually computed.

Gate construction. Compute the generating function exactly to a fixed depth, verify the congruence predictions on their arithmetic progressions and the modular equation coefficient-by-coefficient. Depth is the open question for D-gates: 81 points carried no information; 6,747 did (`docs/GATE-BEFORE-PROVE.md`).

Worked example. PDN1 (partition-diamond) congruences, Du--Yao, arXiv:2503.00004 (`docs/audits/pdn1.md`). Exact series of q_2^2/q_1^5 through q^{500} ($\text{PDN1}(2) = 18$); Theorems 1.1--1.2 congruence predictions on stated APs mod $5^\alpha, 7^\alpha$; modular equation (3.13) with the paper's σ_i polynomials through degree 80 - maximum absolute difference 0 across 6,747 divisibility points. **Verdict:** **PASS (escalate)** on the type-D route; type G not attempted.

E - CAS-transcript replay (CAS = computer algebra system)

Route attacked. Computer-assisted proofs whose decisive steps live inside a CAS session: guessed recurrences, Ore-algebra Grobner bases, creative telescoping certificates. The transcript is the proof, and it is usually unpublished.

Gate construction. Replay the transcript. When the tooling is not available on the campaign stack, record blocked - do not invent operators.

Worked example. q-TSPP, the $q=1$ case of Koutschan's proof (arXiv:0906.1018; docs/blueprint/qltspp-q1.md). The paper's Sec. 4.3 prints only a factorization of the order-7 recurrence's leading coefficient, not the operator; the ∂ -finite description (65 guessed recurrences, 5 MB) was never published, and the recovered notebook's 13 MB diagonal operator needs 3 GB RAM plus the HolonomicFunctions package - unavailable. The campaign's answer is a parametric Lean formalization of the closing argument (DiagonalCertificate, kernel-checked, sorry-free) modulo coefficient recovery, plus author contact for the true coefficients. Type E is the attack type the campaign documents as tooling-limited: the route exists before the infrastructure does.

F - Counterexample search

Route attacked. Universal claims over finite combinatorial objects. The classic BREAK engine: one witness ends the argument.

Gate construction. Exhibit the witness explicitly; close both sides - the upper bound by construction, the lower bound by exact (branch-and-bound or exhaustive) search. The control runs the same solver on graphs where the answer is known both ways, proving it is not an always-fire detector.

Worked example. Baste domination (docs/audits/baste-domination.md). Claim: every finite regular graph of positive degree satisfies $\gamma(G) \leq \gamma_e(G)$. False already at $\Delta = 3$: a 50-vertex witness with $\gamma(G) = 16 > 15 = \gamma_e(G)$, both sides closed by exact search. Control: the solver finds small dominating sets when they exist (K4 at budget 1; Petersen at budget 3) and declines when they don't (Petersen at budget 2) - and the control caught a real bug (n = NUM_VERTICES hardcoded) the moment it tried a non-target graph. **Verdict: BREAK.** (Same mechanism: Chung--Graham--Spiro; Cohen subadditivity, witness (31, 3928) with 500 random pairs showing zero violations elsewhere.)

G - Logical-gap exposure

Route attacked. Proofs whose key step is a structural premise about an algebraic object - the exact lemma the route needs, falsified at the precise failure point.

Gate construction. Isolate the premise; exhibit the smallest instance where it fails; name the failure point. The blueprint header for G-audits reads "Claim (route, not the theorem)."

Worked example. Lame 1847 (docs/blueprint/lame-1847.md). The route needs $\mathbb{Z}[\zeta_p]$ to be a unique factorization domain for the prime at hand. The first prime conductor with class number > 1 is $p = 23$, where $h(\mathbb{Q}(\zeta_{23})) = 3$. The gate checks the minus part: $h_p^- = 1$ for every prime $p < 23$ by the Maillet/OEIS determinant formula, and $h_{23}^- = 3$ (scripts/gates/lame_h23.py); the plus parts $h_p^+ = 1$ for $p < 23$ are cited as external input from classical tables, not recomputed by the gate - the gate's meta records "h_plus_cited": 1 at $p = 23$ and sets $h = h^- \cdot h^+$. So 23 is exactly the point of failure. FLT itself stands; the route dies at $p = 23$. **Verdict: locked PASS** on $h(\mathbb{Q}(\zeta_{23})) = 3$ - the confirmed fact that refutes the route (Sec. 2 polarity rule).

4. The evidentiary disciplines

Sec. 3's attack types are only as honest as the gates that implement them. A gate is a piece of software written by people who already believe the target is fragile; every discipline below exists because the campaign caught itself - or was caught - cutting a corner it had sworn not to cut. Each is stated as a rule, with the failure mode it guards and the incident that earned it.

4.1 Paper-first: pin and read the actual paper

No gate is built from a secondary summary. The procedure is mechanical: fetch the version of record, record its SHA-256 and byte count, extract the text, and read the argument before deciding what the load-bearing step is. The dossier that supplied our targets (`corpus/`) finds targets; it does not reliably describe the mathematics.

Three incidents in the campaign's first days are why the rule admits no exceptions. The dossier described the Frankl union-closed target as an entropy-method argument by "S. Schage" - the paper is a combinatorial deletion-sequence argument by Roberto Demontis. It described the Goldbach semi-continuous (S.C.E.) target at v2 with a "master inequality (24)" - the live paper is v5, its dominant case resting on admitted-unproven inequalities. It described the Legendre target at v2 with a "Theorem 2.18" - the live paper is v4, decisive step Proposition 2.18. In each case a dossier-built gate would have tested the wrong object - the Sarkozy failure mode, named for the campaign's own early mistake of aiming a gate at a paraphrase rather than a lemma. The dossier's paraphrase had Sarkozy's threshold as $|A| \geq cp$; the conjecture as stated reads $|A| > (1/2 - c)p$ - under the paraphrase the claim fell to any small set and the BREAK was vacuous. The retrofit caught it.

4.2 Discrimination control: a check that cannot fail is not evidence

A discrimination control is required where it bites: whenever the verdict is an existence or non-existence claim resolved by search or sampling. Such a verdict can be produced by a search that is too narrow, too broad, or aimed at the wrong object, and none of those failures announces itself - everything the campaign has learned from controls came from this class. The control runs the gate in the opposite direction on a matched input where the opposite verdict is the known-correct answer: for a PASS, perturb the claim and confirm the check rejects the near-miss; for a BREAK, supply a regime where the refuted claim is true and confirm the gate declines. Matched and single-variable - a control that varies two things at once reads like evidence and is worse than none.

Where the verdict is an exact equality or divisibility between two independently computed objects, a discrimination control is ceremony: any discrepancy is caught by construction. What needs auditing there instead is implementation equivalence (does the fast path reproduce the exact oracle? see `pdn1`'s fast-path self-test) and depth (is the sample large enough to carry information? `pdn1` at 81 points was not; at 6,747 it is).

Every control outcome recorded in a receipt JSON is embedded in its gate's own meta JSON, under the `controls` key - control name, verdict, ok, receipt path, receipt SHA-256, and receipt timestamp - by `scripts/gates/record_controls.py` on every aggregate run. A gate whose meta carries `"controls": []` has no *separate* control receipt: that is the honest record for the four exact-equality exemptions and for the two inline controls (`gb_sce`, documented in its audit note; `mah_3`, inside the gate).

The controls have caught real bugs, not hypothetical ones. The Baste counterexample gate's control exposed a hardcoded `n = NUM_VERTICES` that would have silently narrowed the search. The Goldbach S.C.E. gate ran Dusart's inequality, the paper's own Teeter bounds, and its exact model identities through the same gate - PASS on all three - while the three unproven inequalities fired BREAK, and its partition counts reproduced the known Goldbach totals (127, 810, 5402 at $10^4/10^5/10^6$). The Mahler counting-lemma gate's wrong- θ control visibly breaks the bound the right- θ setting confirms. And the discipline cuts the other way too: when the Legendre target's residue turned out to be asymptotic rather than a displayed inequality, the honest move was a prose audit, not a forced numeric gate - a gate that cannot be calibrated is not run.

4.3 Gate before prove: no formalization without a numeric gate

No Lean formalization begins until the claim it targets has survived a numeric gate, and formalization targets carry a no-sorry, kernel-checked standard. The gate decides *whether* the statement is worth proving; the prover then decides whether the proof is correct. Conflating the two - formalizing a statement nobody has checked - is how effort gets spent on false lemmas.

The q-TSPP line follows this order: the milestone-1 Stembridge shakedown (sorry-free, kernel-checked) preceded the milestone-2 diagonal identity, and milestone 3 is blocked on recovering the recurrence coefficients from the authors rather than inventing them. The discipline also sets the boundary of what formalization is *for* here: it is not applied to BREAK verdicts at all - a refuted route needs no formalization of its false lemma.

4.4 Fail-closed replay: hash-pinned inputs, any mismatch fails the lane

Replay lanes recompute from pinned inputs in shards; any checksum mismatch, row disagreement, or unreproducible step fails the lane - no "close enough," no partial passes. The Gomila Λ -bound finite lane verified 3,149,013/3,149,013 rows across 15 shards checksummed at the pinned upstream commit; the Dini, barrier, and tail lanes each carry their own sealed logs. Fail-closed is what makes a PASS verdict mean something: it is the reason the Jacobian-2D computational PASS and the Mahler counting-lemma gate PASS can be cited without hedging. (The Mahler *claim* is a different matter - Sec. 6.2: the gate passed, the claim as a whole is SKIP.)

4.5 Independent anchors and dual implementation: two layers, not one

A single implementation can be wrong in ways its own tests cannot see. The campaign therefore uses two distinct layers. First, **brute-force anchors**: tiny, obviously-correct computations at small parameters that the real implementation must match. The Polya sieve's $p=2$ bug - the 2-adic inverse does not exist for $p=2$, corrupting $L(100000)$ from -288 to -2074 - was caught by brute-force anchors before a second implementation existed. Second, **dual implementation**: a second implementation, written without reading the first, which must agree on every certificate - 91/91 chunks for Polya. The anchor catches the bug class "wrong algorithm, confidently executed"; the dual implementation catches the class "right algorithm, wrong code". One layer is a precaution; two is evidence.

4.6 Durable job discipline: log to disk, mark the exit, verify by hand

Long-running jobs write their output to a durable log under the workspace, append an explicit exit marker, and are verified by reading the log - never by trusting a completion notification. This rule was written after a background dependency-update job died silently and cost an hour before anyone noticed the build directory was empty. Every multi-hour sieve, Lean build, and replay lane in the campaign follows it.

4.7 Where the method failed, and what caught it

The credibility backbone of this paper is not the disciplines but the record of their violations (sources: docs/WORKPLAN.md checkpoints).

- **Paper-first: seven provenance failures, one rule.** Before the rule existed, targets #1--#3 - `cohen_subadditivity`, `baste_domination`, `sarkozy_sum_product` - were gated from the corpus doc plus an abstract fetch, no PDF pinned; all three were retrofitted with the real PDFs on 2026-09-21, and the retrofit caught Sarkozy's misstated threshold (Sec. 4.1). The same day, four more provenance failures in one session: `tang_zhang_schatten` and `thakur_carlitz` (corpus text image-corrupted -> real PDF fetch), `chung_graham_spiro` (ghost entry - table row, no body section, no arXiv ID -> located by live web search), and NCI (the corpus claimed a "Verifiable Gate" the real paper, a first-moment

existence proof, could not support -> SKIP). The rule now opens the protocol document (docs/GATE-BEFORE-PROVE.md), and every audit note since records its paper pin.

- **Ghost corpus entries.** Dossier table rows with no body section - claims about claims with nothing behind them. Corpus hygiene is now part of target intake.
- **Fabricated or garbled identifiers.** A "Reed/Zenodo γ " and a "Sun/Zenodo Catalan" entry turned out to reference nothing retrievable. Identifiers are now resolved to a fetchable artifact before a target is accepted. The rejected "Sun/Zenodo Catalan" entry is not the Sun preprint the catalog later gated: the live type-A BREAK target is Sun's arXiv:2609.04176v1, fetched from arXiv and pinned by SHA-256 - a different artifact from the unresolvable Zenodo handle.
- **The odd-zeta verdict correction (2026-09-20).** An early " Λ_m unevaluable" verdict was superseded by the Lemma 5.1 BREAK; both remain visible, the prior marked superseded, not deleted.
- **A harvested Lean snippet was itself buggy.** A third-party-LLM proof fragment proposed as the Suman gate carried the empty constraint $1 \leq k \wedge k \leq 0$. Harvested artifacts are inputs to be gated, never components to be trusted.
- **Harness bugs caught by controls and anchors,** summarized here because each one justifies a discipline above: the Pochhammer negative-index convention (type C), the hardcoded `n = NUM_VERTICES` (type F), the $p=2$ sieve inverse (Sec. 4.5).

None of these were caught by peer review, because none of them were visible to peer review. They were caught by replaying things - which is the entire argument of this paper.

5. Case studies

The playbook is only as good as its hardest cases. Four studies, one each for a refutation, a confirmation, a canonization, and an unfinished line - each mapped to the Sec. 3 attack type and the Sec. 4 disciplines it leans on. Full records live in docs/audits/; what follows is one page each.

5.1 Kempe--Fritsch: the refutation case (BREAK)

In 1879 Alfred Kempe published a proof of the four-color theorem; in 1890 Heawood found a map that broke it. The campaign's gate does not replay Heawood's map. It replays the modern refutation of Kempe's *algorithm*: the 9-vertex, 21-edge Fritsch graph (Fritsch & Fritsch 1998), transcribed from Gethner et al. Figure 3 (docs/audits/kempe-fritsch.md ; results/kempe_fritsch_gate_meta.json). The load-bearing lemma is that Gadget $\mathfrak{F}_2$'s two successive Kempe-chain switches commute - that the switch order cannot affect the outcome. On the pinned pre-coloring, order A (chain(2,G,Y) then chain(3,G,B)) succeeds while order B (chain(4,G,B) then chain(2,G,Y)) re-introduces G at vertex 8 and tangles vertex 1 irrevocably. The lemma is false, mechanically.

The discrimination control is a matched near-miss: the same Configuration 2 color pattern on a bare 5-wheel, where the two Kempe chains are isolated single vertices and both switch orders agree - confirming the tangle comes from the Fritsch graph's long-range edges (3--7, 4--7, 7--8), not from the machinery being order-sensitive. A separate brute-force properness check after every switch, independent of the gate's own, rules out a fabricated tangle.

Kempe is in this paper as calibration, not as a trophy - the mathematics was settled in the 1890s, and Heawood's map is the historical context, not the witness. Its lesson is the one the whole campaign is built on: a proof can be accepted for a decade and still die at one finite residue, and "the local case is clear" is a fragility signature, not a proof. It exercises paper-first at its most literal: the gate pins Gethner et al. (Involve 2009) - Fritsch & Fritsch 1998 is background, not pinned, and not anyone's summary.

5.2 Gomila Λ -bound: the confirmation case (audit PASS)

Jude Gomila's $\Lambda \leq 0.1787854$ claim was replayed in four lanes and passed all four: 3,149,013/3,149,013 finite rows across 15 checksummed shards; 4/4 Dini legs sealed; 883/883 barrier prisms closed; 36/36 tail checks at 256 and 512 bits ([docs/blueprint/gomila-lambda.md](#)). Every lane is fail-closed (Sec. 4.4): any mismatch would have failed the lane, and the sealed logs are re-verifiable.

Lineage: Polymath15's Theorem 1.2 gave the method and the then-best bound $\Lambda \leq 0.22$; Platt--Trudgian pushed it to 0.2; Gomila's $0.1787854 - t_0 + y_0^2/2$ with $t_0 = 129/800$ and $y_0^2 = 87677/2,500,000$, i.e. 893927/5,000,000 - instantiates Polymath15's machinery at an exact rational parameter row. The audit's question was whether this instantiation's ~3.15 million certificates check out. They do.

This case study exists to show the campaign is not a refutation machine - but it must be labeled for what it is. Every lane executed the audit repository's own verifiers at the pinned commit ([a74738d](#)), on freshly downloaded shards whose checksums were verified first. That is independent *execution* of the author's checker under a hostile prior, not an independent *reimplementation* of the mathematics: no discrimination control is recorded, so the verdict is execution-verified at the pinned commit, not replay-confirmed. The one fully independent check in the case is the Platt--Trudgian margin, recomputed from the primary source (arXiv:2004.09765, Theorem 1): the RH verified height 3,000,175,332,800 minus half the candidate's required height (6,000,000,185,827 / 2 = 3,000,000,092,913.5) leaves 175,239,886.5 exactly. A methodology that cannot confirm is not an audit - but a confirmation must say what kind of confirmation it is.

5.3 Polya's conjecture: the canonization case (BANKED, off the lock)

Polya's conjecture ($L(n) \leq 0$ for all n , L the Liouville summatory function) was disproved by Haselgrove in 1958, but no *checkable* record of the smallest counterexample existed in one place. The campaign banked one: $n = 906,150,257$ - Tanaka's 1980 value, recomputed by two independent sieves - with 91 SHA-256-pinned chunk certificates, a 370/370 verifier, and a Lean formalization of the finite slice through 100 ([docs/audits/polya.md](#)).

Polya is the case study for Sec. 4.5: the $p=2$ sieve bug was caught by brute-force anchors before the dual implementation existed, which then agreed on 91/91 chunks. Neither layer alone is evidence; together they are why the banked counterexample can be cited. It sits off the verdict lock deliberately: canonization is infrastructure, not a verdict about a route.

5.4 q-TSPP: the unfinished line (in progress)

The q-TSPP audit - formalizing the totally symmetric plane partition identities - is the campaign's correct-proof infrastructure build, not a BREAK hunt. The lineage: Okada supplied the determinant reduction to guessed normalized cofactors, and Koutschan's $q=1$ and q -case algorithmic proofs and certificates are the formalization targets. Milestone 1 - the $q=1$ Stembridge shakedown - closed with a sorry-free, kernel-checked recurrence-uniqueness lemma; milestone 2 closed with a parametric order-7 diagonal certificate (8,656/8,656 jobs, zero errors). Milestone 3 is blocked on recovering the explicit $q=1$ recurrence coefficients: the paper does not print them, the thesis does not yield them, and the author's site survives only on Wayback. The author has been contacted; the line waits on his reply.

It belongs in this paper as "gate before prove" (Sec. 4.3) at a projected 6--10-week scale (the blueprint's estimate, not a completed line) - with a stated exception. No q-TSPP gate is on the verdict lock, and no off-lock numeric gate preceded the Lean work: this line formalizes a known-correct proof as infrastructure, so the gate-before-prove rule - which exists to stop the campaign formalizing a claim that might be false - has nothing to guard against. The $q=1$ shakedown, kernel-checked against the known result, is the empirical anchor in place of a gate. The blocker - the

unrecovered recurrence coefficients - is stated plainly instead of being worked around. It is the honest boundary of what the campaign could do alone.

6. Limitations

This section states what the playbook cannot do. Each limitation is one the campaign actually hit, not a hypothetical.

6.1 Replay cannot catch specification errors on its own

A gate replays the computation the paper describes. If the paper describes the *wrong computation* - proves a statement adjacent to the one it claims - the gate passes and the error survives. The campaign's first GAP is the example (`docs/audits/gamma-aejonanonymous.md`): the audited formalization proved `~is_rational_gamma` - a statement about its own predicate, not the irrationality of mathlib's γ . The computation was correct; the specification was wrong. A separate *statement-fidelity* audit - checking that the formal statement says what the paper claims it says - is a partial remedy. It is not a closed gap: fidelity audits are manual, unglamorous, and easy to skip, which is exactly why the error class survives.

6.2 Nonconstructive arguments force SKIP

Where there is no witness to exhibit and no computation to replay, there is no gate. Nonconstructive existence proofs, pure compactness arguments with no extractable bound, and routes whose decisive step cannot be instantiated at any parameters end in SKIP (Sec. 2) - honestly, but unavoidably. The Mahler 3D audit is the boundary case: every finitely checkable layer passed, the prose audit found no defect, and the connectedness step that carries the conclusion has no finite residue at all. The gate verdict stays PASS - it correctly reports what the counting lemma gate checked - but the claim as a whole is SKIP: the decisive step cannot be instantiated at any parameters, so there is nothing to promote. The target is set aside - the correct output of the method, and its admission of reach.

6.3 The certificate cost curve is unsolved

GB-scale certificates against kernel checking remain an unsolved tradeoff. The q-TSPP certificates are hundreds of megabytes - uncheckable inside a proof kernel - so the campaign checks them outside the kernel and formalizes the *shape* of the argument instead. That is a principled compromise, not a solution: trust migrates to the external checker, exactly the kind of software this paper argues should be distrusted. Verified meta-level checkers are the open problem; until they exist, large-certificate proofs get a weaker standard, stated plainly.

6.4 Type E remains tooling-blocked

Type E - replaying a paper's "by symbolic computation" steps - is the attack type the campaign is worst equipped for. The tooling (verified computer algebra, Ore algebras, q-Zeilberger infrastructure) does not exist at the needed level; building it is a multi-year project, not an audit step. The honest response: mark the limitation, record what was checked by other means, and do not pretend a partial replay is a full one. (The JAC-2D gate's D+E typing in Table C1 does not contradict this: its symbolic component re-derived the coefficient formulas from scratch with generic coefficients - it did not replay a CAS transcript, which is what remains blocked.)

6.5 Selection and window

The targets were harvested for fragility, not sampled, and the campaign window is six calendar days, 2026-09-18 to 2026-09-23 (Sec. 1). Nothing in this paper estimates the base rate of defective proofs. The method is reproducible; the results are not a survey. The q-TSPP certificate-rot episode (Sec. 1, Sec. 5.4) is reported as a single instance; whether disappearing certificate archives are typical is a question for a larger sample, not a claim of this paper.

6.6 No pre-publication author notification

The campaign does not notify authors before publishing a verdict, and does not intend to adopt notification as a regular policy. The reason is throughput, not hostility: at twenty-five gates in six days, individualized pre-publication correspondence is infeasible, and a notification rule honored selectively would be worse than none. The substitute is the public record itself. Every BREAK names the lemma, exhibits the false instance, and pins the inputs; every audit note is a document an author can answer point by point, and the repository is the correction channel - a verdict met with a correct counter-argument is superseded in the open, under the append-only rule (Sec. 2). What the campaign does not do is litigate verdicts privately before publishing them. (The q-TSPP author was contacted, but to request missing coefficients for a formalization built on the authors' work, not to disclose a finding.)

7. Related work

This paper claims a playbook and an audit log, not a new formalism. Every piece it assembles exists in several literatures; what follows is where each piece comes from and what this paper does differently.

The computational bound as a genre: Polymath15. The closest existing precedent for the *target* of a replay audit is Polymath's de Bruijn--Newman project (arXiv:1904.12438; Res. Math. Sci. 6 (2019)): a published, computer-assisted bound ($\Lambda \leq 0.22$) whose verification story is "the estimates are proved, the numerics were run by the authors." Its Table 1 is the archetypal replay-audit artifact: a conversion table from "RH verified to height H" into a Λ upper bound, i.e., a published claim whose truth depends on someone else's computation. The difference is posture: Polymath15 *produced* the computation; a replay audit replays someone else's under a hostile prior. Platt and Trudgian (arXiv:2004.09765; Bull. London Math. Soc. 53 (2021), 792--797) then demonstrated that computational claims compose: their verified RH height (3,000,175,332,800) turns Polymath15's table into $\Lambda \leq 0.2$ - the bound is only as strong as someone else's replay of the underlying computation. They also model the honest-boundary behavior Sec. 6 advocates: the next table entry would give $\Lambda < 0.19$, which their height does not reach - "We have not pursued this."

Verification institutions: mathlib review. The strongest existing institution for the thing this paper says is missing is Lean's mathlib - except that it verifies a different object. Mathlib's PR review is post-kernel human review: the kernel guarantees logical correctness, and human reviewers check fitness (naming, generality, placement, faithfulness to the intended statement). A replay audit is the mirror image: for computational claims in published prose there is no kernel, so the audit must supply *both* the correctness check and the fitness check (is this the claim the paper actually makes?). What this paper is not proposing is another formal library with a review queue; the gap is that the computational half of published proofs has no kernel and no review queue.

Proof repair: the mechanized cousin of GAP. Ringer et al. ("Proof Repair across Type Equivalences," PLDI 2021) study what happens *after* a formal proof breaks: tools that repair proof terms across type changes, with the kernel re-checking the repaired artifact. That literature shows what "repairable" means when verification is mechanized - and throws the informal case into relief. This paper's GAPs are prose verdicts about informal arguments with no kernel to confirm a repair, which is why they carry their own evidentiary standard (verbatim quotation, independent re-reading, explicit repairability judgment).

Post-publication scrutiny that does not dispose. PubPeer-style commenting finds problems at scale: Ortega and Delgado-Quiros (EPI, 2023) report that of 17,244 PubPeer-commented articles, only 21.5% of those deserving an editorial notice were ever corrected by the journal. Scrutiny detects; it does not *dispose* - findings sit in comment threads without verdicts. The disposition taxonomy (Sec. 2) is the move from "someone commented" to "a gate fired and the lock recorded it," with PASS carrying the same weight as BREAK so the system cannot become a pure refutation machine.

Adversarial scrutiny in security. Institutionalized adversarial scrutiny exists in industry practice. Competitive audit contests (Code4rena, Sherlock) run many hostile strangers over a code snapshot with multi-stage judging, deduplication, and fix review; professional cryptographic reviews (e.g., NCC Group's) inspect source against a pinned commit and publish findings with severity and exploitability. The judging and fix-review stages are the analogue of this paper's controls and verdict-lock discipline; the pinned-commit report header is the model for the artifact pinning Sec. 1 demands of itself. The difference: security audits judge code against a threat model, while replay audits judge a proof against its own claims - and a BREAK must name the lemma and exhibit the false instance, not merely assign a severity.

The replication movement. Empirical science learned - expensively - that published results often do not survive replay, and built preregistration, replication studies, and adversarial collaboration in response. Mathematics has largely exempted itself on the grounds that proofs are self-verifying. They are, where the proof *is* the reasoning. The verification gap (Sec. 1) is the observation that, increasingly, part of the proof is a computation - and computations are empirical claims about what a machine did. This paper ports the replication movement's core insight (trust the replay, not the report) to the part of mathematics that is actually software.

Machine-checked proofs and the verification filter. Tao ("Machine-Assisted Proof," *Notices of the AMS* 72 (1), Jan. 2025) states the principle that bounds this paper's claims: powerful but unreliable tools are safe when their outputs can be independently verified, suspect otherwise. A PASS from a replay audit is that filter applied to someone else's computation; it is not an endorsement of the claim's importance, just as a BREAK is not a contribution to the field.

What none of these currents supply, to our knowledge, is the combination this paper documents: a fixed disposition taxonomy with a CI-enforced verdict lock (Sec. 2), a mechanism-classified attack-type repertoire (Sec. 3), and a published log of the method's own failures (Sec. 4.7). If that combination exists elsewhere, we would like to cite it.

8. Conclusion

The playbook is the product. Every pattern in Secs. 2--5 - the five dispositions and the rules that keep them from drifting, the mechanism-classified attack types, the evidentiary disciplines and the log of their violations - is reusable against the next claim, by anyone, without our involvement. That is the test this paper sets for itself: a reader who disagrees with every verdict in the audit catalog should still be able to run every gate, check every control, and re-derive every disposition from the pinned artifacts.

What the campaign found, in brief: eight live-literature gates refuted their target - one against a claimed proof (Sun's Catalan route), seven against stated conjectures (Baste et al., Tang--Zhang, Chung--Graham--Spiro, Sarkozy, Thakur, Salez--Youssef, Cohen); where a prior published refutation exists it is cited, and the gates compute their witnesses from the pinned claim artifact. Three more routes fell to prose audits that no gate could reach (Frankl union-closed, Legendre--Newman, Erdos--Straus Theorem 10). Three live claims survived their gates outright - Lopez's Erdos--Straus system, the Lau--Ono and Huang--Lau--Ono--Paule q -expansions, and Du--Yao's partition congruences - and a fourth, the 3D-Mahler claim, passed its counting-lemma gate but stands as claim-level SKIP. Su's low-stakes 2D-Jacobian route survived both its computational gate and an off-lock type-G prose audit, and the off-lock Gomila confirmation was execution-verified at the pinned commit under the same fail-closed disciplines. The method's own failures - abstract fetches gated as papers, ghost corpus entries, a corrected verdict, harness bugs caught by controls - are part of the record, because a methodology that hides its failures cannot be trusted about its successes.

The open problems are stated in Sec. 6 and not repeated here, except the one that matters most: the certificate cost curve. Until large computational certificates can be checked inside a proof kernel at reasonable cost, the strongest computer-assisted proofs will rest on external checkers - software of exactly the kind this paper argues should be distrusted. Closing that gap is the work that would make this playbook obsolete. We would welcome it.

AI Utilization Statement

The candidate-harvest list was assembled with Kimi 3 and Grok 3.1 deep research; gate execution, audit-note drafting, and manuscript preparation were performed by Muse Spark agents at the author's direction. Multiple rounds of pre-submission internal-consistency review of this draft were performed by Claude Opus 5.5 (Anthropic), which cross-checked disposition counts, stratum totals, attack-type labels, and numerical values across sections, tables, and references, and flagged contradictions for the author to resolve. Human-review protocol: the author directed a clean re-run of all 25 locked gates (25/25 [ok], zero drift) and read every gate script, audit note, and prose disposition before this draft.

References

Agama, T. "A proof of the twin prime conjecture." arXiv:1707.03265v4. The twin-prime area-method claim whose Theorem 2.3 fixed constant the fringe gate refuted (BREAK, Table C1).

Code4rena. Audit contest reports. <https://code4rena.com/reports>. Competitive, time-boxed audit contests: multiple Wardens review a code snapshot; findings are judged, deduplicated into unique vulnerabilities, severity-rated, and collected in a final report.

Chattopadhyay, A. "On the Irrationality of the Odd Zeta Values." Preprints.org 202601.1609.v1. doi:10.20944/preprints202601.1609.v1. The odd-zeta irrationality claim refuted by the type-A gate (BREAK, Table C1); the pinned PDF's SHA-256 matches the gate's recorded value.

Du, J. Q. D., and Yao, O. X. M. "Congruences modulo arbitrary powers of 5 and 7 for Andrews and Paule's partition diamonds." arXiv:2503.00004. Source of the PDN1 congruences and modular equation replayed by the type-D gate (Sec. 3).

Gethner, E., Kallichanda, B., Mentis, A. S., et al. "How false is Kempe's proof of the Four Color Theorem? Part II." *Involve* 2(3) (2009), 249--265. doi:10.2140/involve.2009.2.249. The modern telling of Kempe's argument and Heawood's refutation pinned by the Sec. 5.1 gate.

Heawood, P. J. "Map-Colour Theorem." *Quart. J. Pure Appl. Math.* 24 (1890), 332--338. The original refutation of Kempe's 1879 proof: a single map on which simultaneous Kempe-chain interchanges conflict.

Jana, A., and Karmakar, L. "Generalizations of two hypergeometric sums related to conjectures of Guo." arXiv:2501.10109 [math.NT]. The WZ-certificate pair audited clean by the type-C gate (Sec. 3).

Koutschan, C. "Eliminating Human Insight: An Algorithmic Proof of Stembridge's TSPP Theorem." arXiv:0906.1018 (2009). The $q=1$ case of the q -TSPP proof formalized in the campaign's Lean shakedown, the cited version for the $q=1$ order-12 recurrence (Sec. 5.4).

Lean Community. "How to contribute to mathlib" and "Reviewing a mathlib PR." <https://github.com/leanprover-community/leanprover-community.github.io> PRs require passing CI to enter the review queue; reviewers comment and may mark a PR awaiting-author; maintainers give final approval. Review checks placement, duplication/generality, imports, naming, and maintainability.

NCC Group. Security and cryptographic review services. Representative of professional review practice: inspect source against a pinned commit, publish findings with severity and exploitability.

Ortega, J.-L., and Delgado-Quiros, L. "How do journals deal with problematic articles. Editorial response of journals to articles commented in PubPeer." *Profesional de la informacion* 32 (2023). doi:10.3145/epi.2023.ene.18. Dataset of 17,244 PubPeer-commented articles; only 21.5% of articles judged to deserve an editorial notice were corrected by the journal.

Petkovsek, M., Wilf, H. S., and Zeilberger, D. $A = B$. A K Peters, Wellesley, 1996. The Wilf--Zeilberger certificate method: a rational function $R(n,k)$ certifying a hypergeometric identity, the object the type-C gate re-verifies (Sec. 3).

Platt, D., and Trudgian, T. "The Riemann hypothesis is true up to $3 \cdot 10^{12}$." *Bulletin of the London Mathematical Society* 53 (2021), 792--797. doi:10.1112/blms.12460. arXiv:2004.09765. Theorem 1: RH verified to height 3,000,175,332,800; Sec. 3.4 turns Polymath15's Table 1 into Corollary 2 ($\Lambda \leq 0.2$) and declines the $\Lambda < 0.19$ entry: "We have not pursued this."

Polymath, D. H. J. "Effective approximation of heat flow evolution of the Riemann ξ function, and a new upper bound for the de Bruijn--Newman constant." *Research in the Mathematical Sciences* 6 (2019), 31. arXiv:1904.12438. Establishes $\Lambda \leq 0.22$ via effective estimates and numerical computation; Table 1 converts "RH verified to height H" into Λ upper bounds.

Ringer, T., Porter, R., Yazdani, N., Leo, J., and Grossman, D. "Proof Repair across Type Equivalences." In *Proc. PLDI 2021*, 112--127. doi:10.1145/3453483.3454033. PUMPKIN Pi: mechanized proof repair across type equivalences for Coq, with the kernel re-checking repaired artifacts.

Sherlock. Audit contest documentation and reports. <https://audits.sherlock.xyz>. Competitive audit contests with multi-stage judging, deduplication, and fix review.

Su, Y. "Generalizations of local bijectivity of Keller maps and a proof of 2-dimensional Jacobian conjecture." arXiv:1603.01867v43 [math.AG]. The 43-version 2D-Jacobian claim that survived the campaign's type-G logical-gap audit (PASS, Sec. 2).

Sun, Z.-W. "Catalan's constant is irrational." arXiv:2609.04176v1. The type-A target whose numeric gate refuted the route (Sec. 3) - the campaign's sole claimed-proof refutation, as distinct from the seven conjecture counterexamples.

Tao, T. "Machine-Assisted Proof." *Notices of the American Mathematical Society* 72 (1) (Jan. 2025). <https://www.ams.org/journals/notices/202501/noti3041/noti3041.html>.

Zeng, Z., Liu, H., and Ratnavelu, K. "A Counterexample to the Tang Zhang Schatten Norm Conjecture and Sharp Positive Results." arXiv:2608.15558. Theorem 1.1: the exact counterexample ($m = n = 2$, $p = 3/2$) the type-A gate independently re-derives (Sec. 3).

Afrasyab, K. "A 50-Vertex Cubic Counterexample to the Domination-versus-Edge-Domination Conjecture." arXiv:2609.10783. The published refutation of Baste et al.'s conjecture (50-vertex cubic graph, $\gamma = 16 > 15 = \gamma_e$) whose witness the type-F gate recomputes from the pinned claim (Sec. 3).

Aliabadi, M. "A counterexample to the Chung-Graham-Spiro gap-set conjecture." arXiv:2609.04473. The published refutation (fails at $l = 4, 9$ in $U_4 \setminus D_4$) recomputed by the type-F gate (Sec. 3).

Baste, J., Furst, M., Henning, M. A., Mohr, E., and Rautenbach, D. Domination-versus-edge-domination conjecture (2019/2020), as stated in Afrasyab arXiv:2609.10783: every finite regular graph of positive degree satisfies $\gamma(G) \leq \gamma_e(G)$. Refuted by Afrasyab; the type-F gate recomputes the witness (Sec. 3).

Chalise, P., Clark, A., and Gnan, E. K. arXiv:2410.13840v2. The displayed evaluation in the proof of Proposition 3.4 broken by the type-A gate (Sec. 3).

Chen, S., Li, Y., Xi, D., and Xu, Z.-F. "The Mahler Conjecture in Three Dimensions." arXiv:2605.09334. Target of the MAH-3 gate; the gate verdict is PASS on the counting lemma, the claim disposition SKIP (Sec. 6.2).

Chung, F., Graham, R., and Spiro, S. Gap-set conjecture (2020), as stated in Aliabadi arXiv:2609.04473: the l -step gap sets of the down-integer/up-integer partition agree for every $l \geq 1$. Refuted by Aliabadi; the type-F gate recomputes the witness (Sec. 3).

Demontis, R. "The union-closed set conjecture is true." arXiv:2405.03731v1. Target of the FRK-UC GAP note (Sec. 2).

Fritsch, R., and Fritsch, G. *The Four-Color Theorem*. Springer, 1998. The 9-vertex, 21-edge graph and the switch-order failure replayed by the Sec. 5.1 gate.

Ghermoul. arXiv:2508.07367v1, equation (35). The Erdos--Straus target broken by the type-A gate (Sec. 3).

Giraudin, D. N. "A counterexample to a conjecture of Thakur on Carlitz-Wieferich primes." arXiv:2607.15305. The published refutation (explicit c-Wieferich prime of degree 5 over F_{19^3}) recomputed by the type-F gate (Sec. 3).

Gnang, E. K. arXiv:2202.03178v3. The displayed congruence in the proof of Lemma 25 broken by the type-F gate (Sec. 3).

Gomila, J. "Riemann lambda 0.1787854." <https://www.judegomila.com/posts/riemann-lambda-0.1787854>. Audit repository: <https://github.com/judegomila/dbn-lambda-01787854-candidate-audit>. The $\Lambda \leq 0.1787854$ claim execution-verified in Sec. 5.2.

Haselgrove, C. B. "A disproof of a conjecture of Polya." *Mathematika* 5 (1958), 141--145. The first disproof of Polya's conjecture; the canonization target of Sec. 5.3.

Huang, Y., Lau, K., Ono, K., and Paule, P. "Algebraic geometric framework of Rogers--Ramanujan identities." arXiv:2608.15219. One of the two type-D PASS targets (Sec. 3).

Ibarra, J. A. "A counterexample to a subadditivity conjecture of Cohen for Sophie Germain cyclic numbers." arXiv:2607.09793. Cohen's Conjecture 66 (subadditivity of C_σ); the type-F gate recomputes the $(m, n) = (31, 3928)$ witness (Sec. 3).

Kempe, A. B. "On the geographical problem of the four colours." *Amer. J. Math.* 2 (1879), 193--200. The original flawed proof; background to the Sec. 5.1 gate.

Koutschan, C., Kauers, M., and Zeilberger, D. "Proof of George Andrews's and David Robbins's q-TSPP Conjecture." arXiv:1002.4384; *Proc. Natl. Acad. Sci.* 108(6) (2011), 2196--2199. The q-TSPP line's target theorem (Sec. 5.4).

Lame, G. Attempted proof of Fermat's Last Theorem by cyclotomic unique factorization (1847). The route broken at $p = 23$ by the type-G gates (Sec. 3).

Lau, K., and Ono, K. "Modularity of Point Counts for the Curves $X^a=Y^b$: New Rogers--Ramanujan Identities." arXiv:2608.05480. The other type-D PASS target (Sec. 3).

Lopez, M. A. "A Complete Congruence System for the Erdos-Straus Conjecture." arXiv:2404.01508. Target of the `es_cover` PASS gate (Sec. 3).

Munch, F. "A counterexample to a conjecture by Salez and Youssef." arXiv:2504.08055. The published refutation (birth--death chains of increasing length) recomputed by the type-G gate (Sec. 3).

Okada, S. "On the generating functions for certain classes of plane partitions." *J. Combin. Theory Ser. A* 51 (1989), 1--23. The determinant reduction the q-TSPP proof builds on (Sec. 5.4).

Salez, J., and Youssef, P. "Intrinsic regularity in the discrete log-Sobolev inequality." arXiv:2503.02793. Conjecture 1, refuted by Munch; the type-G gate's target (Sec. 3).

Suman, S. "A note on the Irrationality of $\zeta(5)$ and Higher Odd Zeta Values." arXiv:2407.07121. The withdrawn irrationality claim broken by the type-B gate (Sec. 3).

Tait, P. G. Claimed proof of the four-color theorem (1884), refuted by Tutte's 46-vertex counterexample (1946); the gate replays Tutte's counterexample (Sec. 3).

Tanaka, M. Computation of the smallest Polya counterexample, $L(906,150,257) = +1$ (1980), per the campaign's Polya blueprint (Sec. 5.3).

Tang, Q. "A counterexample to a conjecture of Sarkozy on sums and products modulo a prime." arXiv:2603.29992. The published refutation recomputed by the type-F gate (Sec. 3).

Zadehgoi Mohammadi, A., and Kolahdouz, M. "Introducing and Applying S.C.E Model Under Dusart's Inequality to Prove Goldbach's Strong Conjecture for 74 Typical Structures out of All 75 Structural Types of Even Number." arXiv:1909.13230v5. Target of the gb_sce BREAK gate (Sec. 4).

Appendices

A. Verdict-log schema and lock mechanics

Every locked gate writes a machine-readable verdict record to `results/<name>_gate_meta.json`. The schema, by example (`results/gb_sce_gate_meta.json`):

Field	Content
<code>target</code> / <code>source</code>	gate id; paper pinned as <code>arXiv:IDvN</code>
<code>local_pdf</code> / <code>pdf_sha256</code>	pinned paper artifact and its hash
<code>lemma</code>	the paper's claim under test, quoted or cited by location
<code>instance</code>	the parameters the gate instantiated
<code>false_instance</code>	the concrete witness (for BREAK)
<code>verdict</code>	<code>BREAK</code> or <code>PASS</code>
<code>not_this_gate</code>	what the verdict does <i>not</i> cover (polarity rule, Sec. 2)
<code>timestamp</code> / <code>elapsed_s</code>	when and how long the gate ran
<code>ok</code>	whether the gate itself executed cleanly
<code>controls</code>	control outcomes embedded in the gate's own meta: list of <code>control</code> / <code>verdict</code> / <code>ok</code> / receipt path / receipt SHA-256 / receipt timestamp; <code>[]</code> where no control receipt exists. <code>receipt_sha256</code> hashes the receipt file at embed time only.

Gate-specific fields (counts tested/violated, margins, control references) extend the schema. The lock's verdict check reads exactly one field from each meta record: `verdict`, compared against `EXPECTED_VERDICT`. `ok` - whether the gate executed cleanly - is conventional rather than enforced: every current record carries it, and the aggregate run reports per-gate `ok` from the process return code, but the lock performs no schema validation over it. Field naming varies per gate and the lock tolerates it: the Kempe--Fritsch record, for example, uses `gate` where the table says `target` and `local_pdf_sha256` where it says `pdf_sha256`, and omits `instance`, `not_this_gate`, and `elapsed_s`. A record missing a conventional field is a documentation gap, not a lock failure; the fail-closed check is the verdict comparison in `check.py`.

The lock itself is `EXPECTED_VERDICT` in `scripts/gates/check.py`: a map from gate id to `(meta-file, expected-verdict)`. The aggregate run executes every gate, compares each recorded verdict against its expectation, and reports `[ok]` or `[DRIFT]` per gate; any drift - in either direction - fails the run. Adding a gate means adding its entry in the same commit as the gate. Removing or re-pinning a verdict requires the re-audit to be recorded first. Prose dispositions (GAP, SKIP, UNKNOWN) and off-lock confirmations are not in the map; they live in `docs/audits/` (Sec. 2).

B. Gate harness conventions

- **Stdlib first.** Gates are dependency-free Python where possible (exact rational arithmetic via `fractions`, sieves by hand); where heavy numerics are needed the dependency is stated and pinned. A gate that cannot run on a fresh checkout is not a gate.
- **Deterministic.** Fixed seeds; no wall-clock dependence; no network access at run time. Inputs are hash-pinned files under `incoming/` or generated deterministically in the script.
- **Self-verifying inputs.** Hardcoded mathematical data (facet lists, group tables) carries its own consistency checks - supporting-plane verification, Euler characteristic, extremality - executed before the gate logic runs.
- **Controls are code, not commentary.** Each gate's discrimination control is a standalone runnable script under `scripts/controls/`, with its own receipt JSON under `results/` - and every control outcome recorded in a receipt is also embedded in its gate's own meta JSON, under the `controls` key (control name, verdict, ok, receipt path, receipt SHA-256, receipt timestamp). The embedding is done by `scripts/gates/record_controls.py`, which the aggregate (`scripts/gates/check.py`) calls after every gate run; gates with no control receipt carry `"controls": []`, an honest empty record rather than an omitted field. Two gates are the documented exception to "code, not commentary": they carry their control inline instead of as a script - `gb_sce` (matched control described in its audit note) and `mah_3` (wrong- θ control inside the gate) - and their metas record `"controls": []`, i.e. no *separate* receipt. Gates in the exact-equality class carry no discrimination control; Sec. 4.2 gives the exemption and its substitutes. Control failures are report-only: a receipt with `ok: false` is embedded as-is, printed loudly, and recorded in `gates_check_meta.json` as `controls_ok: false`, but it does not fail the aggregate or the verdict lock. Controls are re-run manually, outside the aggregate - `recorded_at` is the embed time, not the control-run time, so a receipt can predate its gate's last run. `receipt_sha256` binds the receipt file as of embed time; it binds neither the control script nor the gate code. Appendix C maps every gate to its control.
- **Exit codes mean it.** Exit 0 with `verdict` set: the gate ran and decided. Any exception, missing input, or guard failure (e.g. paper hash mismatch) is an abort, never a verdict.
- **Timestamps refresh; verdicts don't.** The aggregate run rewrites `timestamp` fields across meta files, and reserializes every gate meta (a meta is rewritten only when its embedded controls actually change, so repeat runs produce no diff churn); verdict fields change only by re-audit commit.

C. Audit catalog (locked gates)

Table C1. Gate, target, stratum, verdict, claim-level disposition, attack type, evidence category, and control, from `EXPECTED_VERDICT` (`scripts/gates/check.py`). Strata: hist = historical calibration, live = live literature, fringe = low-stakes preprints, infra = infrastructure. Evidence: proof-route refutation (a claimed proof's route is broken), conjecture counterexample (a stated conjecture is falsified), recomputation (a published refutation's witness is recomputed from the pinned claim artifact), confirmation (the claim survives), infrastructure. Controls are standalone scripts under `scripts/controls/` with receipt JSONs under `results/` (App B); "exact-equality" marks the Sec. 4.2 exemption.

Gate	Target	Stratum	Verdict	Claim disposition	Attack	Evidence	Control
kempe_fritsch	Fritsch & Fritsch 1998 gadget	hist	BREAK	claim BREAK	B+F	recomputation	kempe_fritsch_break_control.py
tait_tutte	Tait 1884 / Tutte 1946	hist	BREAK	claim BREAK	F	recomputation	taite_tutte_break_control.py
lame_h23	Lame 1847 cyclotomic route	hist	PASS	claim BREAK (route)	G	proof-route refutation	not required (exact equality)
lame_ideal_neg23	Lame 1847 ideal variant	hist	PASS	claim BREAK (route)	G	proof-route refutation	lame_ideal_control.py
cohen_subadditivity	Cohen Conjecture 66	live	BREAK	claim BREAK	F	conjecture counterexample	cohen_break_control.py
baste_domination	Baste et al. 2019/2020	live	BREAK	claim BREAK	F	conjecture counterexample	baste_break_control.py
sarkozy_sum_product	Sarkozy Conjecture 65	live	BREAK	claim BREAK	F	conjecture counterexample	sarkozy_break_control.py
tang_zhang_schatten	Tang--Zhang	live	BREAK	claim BREAK	A	conjecture counterexample	tang_zhang_break_control.py
thakur_carlitz	Thakur 2015	live	BREAK	claim BREAK	F	conjecture counterexample	thakur_break_control.py
chung_graham_spiro	Chung--Graham--Spiro 2020	live	BREAK	claim BREAK	F	conjecture counterexample	chung_graham_break_control.py
salez_youssef_logsobolev	Salez--Youssef Conj. 1	live	BREAK	claim BREAK	A	conjecture counterexample	salez_youssef_break_control.py
cat_g	Sun Catalan route	live	BREAK	claim BREAK	A	proof-route refutation	cat_g_break_control.py
es_cover	Lopez Erdos--Straus cover	live	PASS	claim PASS	D	confirmation	es_cover_control.py
rr_qexpand	Lau--Ono; Huang--Lau--Ono--Paule	live	PASS	claim PASS	D	confirmation	not required (exact equality)
pdn1	Du--Yao	live	PASS	claim PASS	D	confirmation	not required (exact equality)
mah_3	Mahler 3D counting lemma	live	PASS	claim SKIP	A+G	confirmation (gate)	inline wrong- θ control
suman_eq48	Suman $\zeta(5)$	fringe	BREAK	claim BREAK	B	proof-route refutation	<code>scripts/controls/break_control.py</code> (shared; receipt <code>suman_break_control_meta.json</code>)
odd_zeta_1609	odd-zeta claim (Chattopadhyay)	fringe	BREAK	claim BREAK	B	proof-route refutation	<code>scripts/controls/break_control.py</code> (shared; receipt <code>odd_zeta_break_control_meta.json</code>)
es5_eq35	Ghermoul ES eq. 35	fringe	BREAK	claim BREAK	A	proof-route refutation	es5_eq35_break_control.py
tpc_area	twin-prime area method	fringe	BREAK	claim BREAK	F	proof-route refutation	tpc_area_break_control.py
tpc_gn	Chalise--Clark--Gnang Prop. 3.4	fringe	BREAK	claim BREAK	A	proof-route refutation	tpc_gn_break_control.py
krr_cl	Gnang Lemma 25	fringe	BREAK	claim BREAK	F	proof-route refutation	krr_cl_break_control.py
gb_sce	Zadehgoi Mohammadi & Kolahdouz	fringe	BREAK	claim BREAK	A	proof-route refutation	audit-note matched control
jac_2d	Su 2D-Jacobian v43	fringe	PASS	claim PASS	D+E	confirmation	<code>jac_2d_break_control.py</code> (discrimination control despite the name; PASS gate)
giuga_oracle	standing Giuga oracle	infra	PASS	n/a	.. ¹	infrastructure	not required (exact equality)
Total: 25 gates		4 / 12 / 8 / 1	17 BREAK / 8 PASS			machine-checked against EXPECTED_VERDICT	

Off-lock dispositions (prose, not on the verdict lock): FRK-UC (GAP, G), LEG-NS (GAP, G), Erdos--Straus Theorem 10 (GAP, G, repairable), JAC-2D type-G (PASS, G), γ formalization route (GAP, off-lock: the route targeted mathlib's `EulerMascheroniConstant` rather than the paper's claim - a statement-fidelity gap, not a refuted claim), Jana--Karmakar type-C (PASS, off-lock: WZ-certificate audit of a published proof, not a fragile route), NCI (SKIP), quantum Hedetniemi (UNKNOWN). Off-lock confirmations: Gomila Λ -bound (execution-verified at pinned commit, Sec. 5.2), Polya (BANKED).

D. Glossary

Attack type. One of the mechanism classes A--G (Sec. 3): the kind of route a proof takes to its conclusion.

BANKED / canonization. An off-lock infrastructure product: a checkable record banked for the community (e.g., Polya's smallest counterexample), not a verdict about a route.

BREAK. Disposition: a specific lemma, as the paper states it, is false, exhibited by a gate. Recorded as lemma, instance, false instance.

Control (discrimination control). A standalone runnable check (Sec. 4.2) showing the gate produces the opposite verdict where the opposite is correct; its outcome is recorded in a receipt JSON and embedded in the gate's meta. Two gates carry their control inline (`gb_sce` , `mah_3`); four exact-equality gates are exempt.

Corpus / dossier / harvest. Target selection: the *harvest* is the fragility-targeted search; the *dossier* is the document it produces; the *corpus* is the dossier directory.

Disposition. An audit's final outcome - exactly one of BREAK, GAP, PASS, SKIP, UNKNOWN - defined by what was done, not by how bad the news is.

Dual implementation. A second, clean-room implementation of a computation, written without reading the first, that must agree on every certificate.

Fail-closed. Any checksum mismatch, row disagreement, or unreproducible step fails the lane; no partial passes.

GAP. Disposition: the paper's argument does not establish its conclusion, but no lemma statement was falsified. Found by prose audit; carries its own evidentiary standard (Sec. 2).

Gate. The executable check: a script plus hash-pinned inputs plus a machine-readable meta record. A gate records a *verdict* (BREAK/PASS); an audit ends in a *disposition*.

Gate-before-prove. No Lean formalization begins until the claim it targets has survived a numeric gate.

Hostile prior. The audit stance: a computational claim is unconfirmed until an independent replay confirms it, regardless of the prose around it.

Lane. A replay execution track - e.g., the Gomila audit's finite, Dini, barrier, and tail lanes.

Paper-first. No gate from a secondary summary: pin the version of record (SHA-256, byte count) and read the argument first.

PASS. Disposition: the gate confirmed what it was built to test - a statement about the audit's reach, not a certificate of truth. Carries the same weight as BREAK.

Prose audit. Manual proof-reading path: GAP analysis and statement-fidelity checks, where no finite residue exists to gate.

Residue. The finitely checkable remainder of a claim - what a gate actually tests.

Route. The specific chain of lemmas under test. Gates refute routes, not theorems.

SKIP. Disposition: the target as stated admits no gate - the verdict is about the audit ("we cannot test this"), not the paper.

UNKNOWN. The checkable parts check out, but the decisive step cannot be replayed on the campaign's toolchain.

Verdict / verdict lock. A gate's recorded outcome, pinned in `EXPECTED_VERDICT` ; the lock runs in CI and any drift fails the build.

E. Notation

Table E1. Symbols used in the manuscript.

Symbol	Meaning
$B(n, k), B(n, n)$	TSPP orbit-counting generating function; the diagonal identity is $B(n, n) = 1$
$C_{p,m}^{TZ}$	Tang--Zhang conjectured Schatten-norm constant
$C_\sigma(N)$	count of Sophie Germain cyclic integers in $[1, N]$ (Cohen)
D_ℓ, U_ℓ	down-integer / up-integer ℓ -step gap sets (Chung--Graham--Spiro)
d_n	$\text{lcm}(1, 2, \dots, n)$ (Suman)
G	Catalan's constant (Sun)
h, h^-, h^+	class number and its minus/plus parts (Lame)
$L(n)$	Liouville summatory function (Polya)
t_0, y_0	Gomila parameter row: $\Lambda \leq t_0 + y_0^2/2$
Λ	de Bruijn--Newman constant
Λ_m	odd-zeta auxiliary sequence
ζ	Riemann zeta function
γ	Euler--Mascheroni constant; $\gamma(G)$ domination number (Baste, context-dependent)
$\gamma_e(G)$	edge domination number (Baste)

F. Execution environment

Gates run on stock Python 3 (standard library first: `fractions`, hand-rolled sieves); where heavy numerics are needed the dependency is stated and pinned in the gate's docstring. No network access at gate run time; fixed seeds; inputs are SHA-256-pinned files under `incoming/` or generated deterministically in the script. The Lean 4 formalization lines (q-TSPP milestones, Polya slice, Cohen ancillary) use Lean toolchain 4.32.2 with pinned mathlib and dependency checkouts; kernel-checked builds report jobs completed and error counts (e.g., 8,656/8,656, zero errors). The verdict-lock aggregate (`scripts/gates/check.py`) runs every gate and compares each recorded verdict against `EXPECTED_VERDICT`; any drift fails the run.

1. `giuga_oracle` is a standing verification oracle (known Giuga composites checked against Korselt), not an attack on a route; it carries no A--G letter. [↗](#)